CN Lab Exam: Q6, Q8, Q10-Q14

Prepared for 11 November 2026. Standard C89 style for an older C compiler.

1. Verification of sir's solutions

The syllabus specifies Q6 classful addressing, Q8 Dijkstra, Q10 distance vector routing, Q11 link state routing, Q12 leaky bucket, Q13 encryption/decryption and Q14 troubleshooting commands. Q7 and Q9 are excluded from this revision.

Sir's supplied solutions cover Q10-Q14. They do not provide Q6 or Q8, so those two programs below are new implementations, not corrections of supplied code.

Q10: Distance vector - correction required

The comparison uses:

```
costmat[i][k] + rt[k].dist[j]
```

but the assignment uses:

```
rt[i].dist[k] + rt[k].dist[j]
```

Use the first expression in BOTH places. `costmat[i][k]` is the direct link cost to neighbour k; `rt[i].dist[k]` may already be a route through another neighbour. Mixing them can make the reported next hop inconsistent with the reported distance.

A verified counterexample:

```
5
0 1 7 10 13
1 0 6 1 12
7 6 0 15 6
10 1 15 0 1
13 12 6 1 0
```

For router 1 to router 5, the original algorithm reports next hop 4 and cost 3. Going directly to router 4 costs 10, then router 5 costs 1: total 11. The shortest route is 1 -> 2 -> 4 -> 5, cost 3, so the correct next hop is 2.

The PDF's three-router sample also has two incorrect printed next hops: router 1 to 3 should go via 2, and router 3 to 1 should go via 2. Running its code, after removing `getch()`, confirms these sample-output errors.

Other details: `getch()` is not standard C and has no declaration in the supplied code. The router array holds only 10 routers despite some other arrays having size 20. The replacement uses consistent size-10 arrays and signed integers with `%d`.

Q11: Link state - basic idea correct, loop needs correction

It uses Dijkstra, which is appropriate for shortest-path calculation in link state routing. However, the source is already marked visited and the program still performs `count` selection iterations. It also has no reliable stop when there is no reachable unvisited router.

On this disconnected graph, with source 0 in the original program:

```
3
0 1 -1
1 0 -1
-1 -1 0
0
```

it prints a path `2 <-- 0` with cost 1000, even though router 2 is unreachable. The replacement stops when no candidate exists and prints `Unreachable` without inventing a path.

The original uses router numbers starting at 0 and converts negative entries to 1000. The replacement consistently uses router numbers starting at 1 and 9999 for no link.

Q12: Leaky bucket - core idea reasonable, changes needed

- `rand()` needs `<stdlib.h>`, which is missing. `<stdbool.h>` is unused and may be unavailable on an old compiler.
- `inputRate` is subtracted from the bucket, so it actually represents the OUTPUT/leak rate.
- `rand() % packetSize` generates a random size from 0 to `packetSize - 1`; it does not send a packet of the entered fixed size.
- `time <= totalTime`, starting at zero, runs `totalTime + 1` slots.
- Output lacks newlines, and the program stops without draining any remaining bucket content.

The replacement accepts arrival sizes as input, reports sent/dropped/remaining data, and drains the bucket after the last arrival.

Q13: Encryption/decryption - works under simple restrictions

Adding a small key to each character and subtracting it again is a basic character-shift demonstration. It works for simple words and suitable keys. It is not an alphabet-wrapping Caesar cipher: `XYZ` with key 3 becomes `[\]`, not `ABC`. Arbitrary shifts can also leave printable character ranges or create string terminators.

The replacement uses uppercase Caesar encryption with wrap-around. This is a chosen simple implementation of the broad syllabus question; the supplied question does not require a named cipher. It is an educational cipher, not secure modern encryption.

Q14: Commands - largely correct

This question needs commands in Windows Command Prompt, not a C program. Use `ping <address>` for a short test. A continuous ping uses `-t` with an ordinary hyphen and can be stopped

with Ctrl+C. The PDF's IP addresses, DNS answers and MAC addresses are examples, not values guaranteed on your computer.

2. Common rules for the simplified programs

- All six programs compiled successfully with strict C89 settings and warnings treated as errors. The actual college compiler was not available to run.
- Declarations are at the beginning of blocks; arrays have fixed sizes. There are no `//` comments, variable-length arrays, `stdbool.h`, or declarations inside `for` headers.
- Each `.c` file is a separate program. Compile and run ONE at a time.
- Use valid, small inputs. There are no repeated input-validation loops.
- Q6: four valid octets from 0 to 255.
- Routing programs: 1-10 routers, diagonal 0, positive link costs 1-500, and 9999 for no direct link. For an undirected graph, make the matrix symmetric. Router numbers in input/output start at 1.
- Do not enter 0 for a missing link: zero would mean a zero-cost link.
- Q12: positive capacity and output rate, nonnegative arrival sizes, small integers.
- Q13: 1-99 uppercase English letters, no spaces, key 0-25.
- `int main(void)` and `return 0;` are valid old standard C. `clrscr()` and `getch()` are unnecessary for the algorithm. In an IDE, use its output-window command if the result screen closes.

Verification included 40 small routing graphs, including disconnected graphs and one-router cases. Distances were checked against an independent Floyd-Warshall calculation; paths and next hops were also checked. Classful addressing passed 12 representative/boundary inputs, Caesar passed all 26 keys, and leaky bucket passed four scenarios.

3. Programs and practice examples

The output blocks show the result portion of actual test runs, with input prompts omitted for readability.

Q6. Classful addressing

Idea: The first octet determines the address class. For A, B and C, the class determines the default mask and the network/host split.

Class	Ordinary first-octet range	Default mask	Network / host octets
A	1-126	255.0.0.0	1 / 3
B	128-191	255.255.0.0	2 / 2
C	192-223	255.255.255.0	3 / 1
D	224-239	Not applicable	Multicast
E	240-255	Not applicable	Reserved

0 and 127 lie in the historical Class A bit range but are special ranges; 127 is used for loopback. The program identifies them separately. Classful addressing is the historical scheme requested in this lab.

Remember: read four octets -> check first octet -> print class, mask and network/host parts.

Code: `q6_classful.c`

```c
#include <stdio.h>

int main(void)
{
    int a, b, c, d;

    printf("Enter IPv4 address: ");
    scanf("%d.%d.%d.%d", &a, &b, &c, &d);

    if (a == 0 || a == 127)
        printf("Special address range\n");
    else if (a <= 126) {
        printf("Class A\nDefault mask: 255.0.0.0\n");
        printf("Network address: %d.0.0.0\n", a);
        printf("Host part: %d.%d.%d\n", b, c, d);
    }
    else if (a <= 191) {
        printf("Class B\nDefault mask: 255.255.0.0\n");
        printf("Network address: %d.%d.0.0\n", a, b);
        printf("Host part: %d.%d\n", c, d);
    }
    else if (a <= 223) {
        printf("Class C\nDefault mask: 255.255.255.0\n");
        printf("Network address: %d.%d.%d.0\n", a, b, c);
        printf("Host part: %d\n", d);
    }
    else if (a <= 239)
        printf("Class D: Multicast\n");
    else
        printf("Class E: Reserved\n");

    return 0;
}
```

Sample input, in the order requested:

```
192.168.1.10
```

Result:

```
Class C
Default mask: 255.255.255.0
Network address: 192.168.1.0
Host part: 10
```

Q8. Dijkstra shortest path

Idea: Start the source distance at zero and all others at infinity. Repeatedly select the nearest unvisited node, mark it visited, and improve distances to its neighbours.

Remember: initialize -> select minimum -> mark visited -> relax neighbours -> print.

- `dist[i]` : current best distance to i.
- `visited[i]` : whether its shortest distance is finalized.
- `parent[i]` : previous node on the route; used to print the complete path.
- `u == -1` : no reachable unvisited node remains; this is an algorithm stop condition.
- `printPath()` prints predecessors first using recursion, then the current node.

The sample compares direct route 1 -> 3, cost 7, with 1 -> 2 -> 3, cost 2 + 1 = 3. Dijkstra requires nonnegative weights.

Code: `q8_dijkstra.c`

```c
#include <stdio.h>
#define INF 9999

int cost[10][10], dist[10], visited[10], parent[10];

void printPath(int v)
{
    if (parent[v] != -1) {
        printPath(parent[v]);
        printf(" -> ");
    }
    printf("%d", v + 1);
}

int main(void)
{
    int n, i, j, s, step, u, min;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter cost matrix (9999 = no link):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    printf("Enter source (1 to %d): ", n);
    scanf("%d", &s);
    s--;

    for (i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }
    dist[s] = 0;

    for (step = 0; step < n; step++) {
        min = INF;
        u = -1;
        for (i = 0; i < n; i++)
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                u = i;
            }
        if (u == -1)
            break;
        visited[u] = 1;

        for (j = 0; j < n; j++)
            if (!visited[j] && cost[u][j] != INF &&
                dist[u] + cost[u][j] < dist[j]) {
                dist[j] = dist[u] + cost[u][j];
                parent[j] = u;
            }
    }

    printf("Node\tCost\tPath\n");
    for (i = 0; i < n; i++) {
        if (dist[i] == INF)
            printf("%d\tUnreachable\n", i + 1);
        else {
            printf("%d\t%d\t", i + 1, dist[i]);
            printPath(i);
            printf("\n");
        }
    }
    return 0;
}
```

Sample input, in the order requested:

```
3
0 2 7
2 0 1
7 1 0
1
```

Result:

```
Node    Cost    Path
1   0   1
2   2   1 -> 2
3   3   1 -> 2 -> 3
```

Q10. Distance vector routing

Idea: Each router improves its route to a destination using a neighbour's distance to that destination.

```
new cost = direct cost to neighbour + neighbour's cost to destination
```

Remember: copy cost matrix -> try every neighbour -> update cost and next hop -> repeat until unchanged.

- `i`: current router; `j`: destination; `k`: possible next-hop neighbour.
- `next[i][j] = k` records the neighbour to send to first.
- `changed` repeats the process until the routing tables stop changing.
- The `cost` matrix stays fixed; the `dist` matrix improves.

This is a static distance-vector simulation in one program; the arrays represent the routers' distance vectors.

Code: `q10_distance_vector.c`

```c
#include <stdio.h>
#define INF 9999

int main(void)
{
    int cost[10][10], dist[10][10], next[10][10];
    int n, i, j, k, changed;

    printf("Enter number of routers: ");
    scanf("%d", &n);
    printf("Enter cost matrix (9999 = no link):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            dist[i][j] = cost[i][j];
            next[i][j] = j;
        }

    do {
        changed = 0;
        for (i = 0; i < n; i++)
            for (j = 0; j < n; j++)
                for (k = 0; k < n; k++)
                    if (cost[i][k] != INF && dist[k][j] != INF &&
                        cost[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = cost[i][k] + dist[k][j];
                        next[i][j] = k;
                        changed = 1;
                    }
    } while (changed);

    for (i = 0; i < n; i++) {
        printf("\nRouter %d\nDestination\tNext hop\tCost\n", i + 1);
        for (j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("%d\t\t-\t\tUnreachable\n", j + 1);
            else if (i == j)
                printf("%d\t\t-\t\t0\n", j + 1);
            else
                printf("%d\t\t%d\t\t%d\n",
                       j + 1, next[i][j] + 1, dist[i][j]);
        }
    }
    return 0;
}
```

Sample input, in the order requested:

```
3
0 2 7
2 0 1
7 1 0
```

Result:

```
Router 1
Destination Next hop    Cost
1       -       0
2       2       2
3       2       3

Router 2
Destination Next hop    Cost
1       1       2
2       -       0
3       3       1

Router 3
Destination Next hop    Cost
1       2       3
2       2       1
3       -       0
```

Q11. Link state routing

Idea: A router with the complete network map runs Dijkstra to build its routing table. This program repeats Dijkstra for every source router, so you see all routing tables.

Reuse Q8: the minimum-selection and distance-update logic are the same. The outer `s` loop changes the source router. Instead of printing full paths, this program stores the first hop.

- If `u == s`, the improved neighbour `j` is itself the first hop.
- Otherwise, reaching `j` through `u` uses the same first hop as reaching `u`, so `next[j] = next[u]`.

The matrix represents an already-known link-state database. This is the shortest-path calculation part of link state routing; it does not simulate advertisement flooding or a full OSPF protocol.

Q10 and Q11 should give the same shortest costs for the same graph, even though they calculate them differently.

Code: `q11_link_state.c`

```c
#include <stdio.h>
#define INF 9999

int main(void)
{
    int cost[10][10], dist[10], visited[10], next[10];
    int n, i, j, s, step, u, min;

    printf("Enter number of routers: ");
    scanf("%d", &n);
    printf("Enter cost matrix (9999 = no link):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (s = 0; s < n; s++) {
        for (i = 0; i < n; i++) {
            dist[i] = INF;
            visited[i] = 0;
            next[i] = -1;
        }
        dist[s] = 0;

        for (step = 0; step < n; step++) {
            min = INF;
            u = -1;
            for (i = 0; i < n; i++)
                if (!visited[i] && dist[i] < min) {
                    min = dist[i];
                    u = i;
                }
            if (u == -1)
                break;
            visited[u] = 1;

            for (j = 0; j < n; j++)
                if (!visited[j] && cost[u][j] != INF &&
                    dist[u] + cost[u][j] < dist[j]) {
                    dist[j] = dist[u] + cost[u][j];
                    if (u == s)
                        next[j] = j;
                    else
                        next[j] = next[u];
                }
        }

        printf("\nRouter %d\nDestination\tNext hop\tCost\n", s + 1);
        for (i = 0; i < n; i++) {
            if (dist[i] == INF)
                printf("%d\t\t-\t\tUnreachable\n", i + 1);
            else if (s == i)
                printf("%d\t\t-\t\t0\n", i + 1);
            else
                printf("%d\t\t%d\t\t%d\n",
                       i + 1, next[i] + 1, dist[i]);
        }
    }
    return 0;
}
```

Sample input, in the order requested:

```
3
0 2 7
2 0 1
7 1 0
```

Result:

```
Router 1
Destination Next hop    Cost
1       -       0
2       2       2
3       2       3

Router 2
Destination Next hop    Cost
1       1       2
2       -       0
3       3       1

Router 3
Destination Next hop    Cost
1       2       3
2       2       1
3       -       0
```

Q12. Leaky bucket

Idea: The bucket stores arriving data. It transmits up to a fixed amount per time slot. An arrival that does not fit is dropped.

Remember: accept or drop arrival -> send `min(bucket, rate)` -> subtract sent amount -> print.

Each entered arrival is one packet's size, in bytes (or consistent data units). Like sir's version, this program drops the WHOLE arriving packet when it does not fit. It does not partially accept a packet. Arrivals are processed before transmission in each slot. Output is modelled as data units leaking at a fixed maximum rate; packet boundaries are not tracked in the stored data.

For capacity 10 and rate 3:

- Time 1: accept 6, send 3, leave 3.
- Time 2: arrival 8 cannot fit beside the stored 3, so drop all 8; send the stored 3.
- Times 5 and 6: no new arrivals; finish draining the bucket.

The rate must be positive so the final draining loop can finish.

Code: `q12_leaky_bucket.c`

```c
#include <stdio.h>

int main(void)
{
    int capacity, rate, n, time, incoming, sent, dropped;
    int bucket = 0;

    printf("Enter bucket capacity, output rate and time slots: ");
    scanf("%d%d%d", &capacity, &rate, &n);
    printf("Time\tIncoming\tDropped\tSent\tRemaining\n");

    for (time = 1; time <= n || bucket > 0; time++) {
        incoming = 0;
        if (time <= n) {
            printf("Enter arrival size at time %d: ", time);
            scanf("%d", &incoming);
        }

        dropped = 0;
        if (bucket + incoming <= capacity)
            bucket += incoming;
        else
            dropped = incoming;

        sent = rate;
        if (bucket < rate)
            sent = bucket;
        bucket -= sent;

        printf("%d\t%d\t\t%d\t%d\t%d\n",
               time, incoming, dropped, sent, bucket);
    }
    return 0;
}
```

Sample input, in the order requested:

```
10 3 4
6
8
4
7
```

Result:

```
Time  Incoming  Dropped  Sent  Remaining
1     6         0        3     3
2     8         8        3     0
3     4         0        3     1
4     7         0        3     5
5     0         0        3     2
6     0         0        2     0
```

Q13. Encryption and decryption

Idea: Caesar cipher shifts each uppercase letter by a key. `% 26` wraps around the alphabet.

```
Encrypt: 'A' + (letter - 'A' + key) % 26
Decrypt: 'A' + (letter - 'A' - key + 26) % 26
```

Remember: convert A-Z to 0-25 -> add/subtract key -> wrap -> convert back.

For X and key 3: X is 23; `(23 + 3) % 26` is 0, giving A. During decryption, `+26` keeps the value nonnegative before `% 26`.

Use only uppercase letters and a key from 0 to 25 for this short version. `HELLO` with key 3 gives `KHOOR`; `XYZ` with key 3 gives `ABC`.

Code: `q13_encryption.c`

```c
#include <stdio.h>

int main(void)
{
    char text[100];
    int key, i;

    printf("Enter uppercase text (no spaces): ");
    scanf("%99s", text);
    printf("Enter key (0 to 25): ");
    scanf("%d", &key);

    for (i = 0; text[i] != '\0'; i++)
        text[i] = 'A' + (text[i] - 'A' + key) % 26;
    printf("Encrypted text: %s\n", text);

    for (i = 0; text[i] != '\0'; i++)
        text[i] = 'A' + (text[i] - 'A' - key + 26) % 26;
    printf("Decrypted text: %s\n", text);

    return 0;
}
```

Sample input, in the order requested:

```
XYZ
3
```

Result:

```
Encrypted text: ABC
Decrypted text: XYZ
```

Q14. Basic troubleshooting commands

Run these in **Windows Command Prompt**, separately. These are Windows forms matching sir's sheet; macOS uses different commands for some tasks. No C program is needed.

Command	Purpose	What to observe
`ipconfig`	Show IP configuration	IPv4 address, subnet mask and default gateway
`ipconfig /all`	Show detailed adapter configuration	Physical/MAC address, DHCP and DNS details
`ping 127.0.0.1`	Test the local IP stack	Reply and packet-loss statistics; this does not prove Internet access
`ping mgit.ac.in`	Attempt to resolve the name and test ICMP reachability	Replies or timeouts and summary statistics
`tracert mgit.ac.in`	Trace responding hops toward a destination	Router hops and round-trip times
`nslookup mgit.ac.in`	Query DNS	DNS server used and returned address(es)
`arp -a`	Display the ARP cache	Local IPv4-to-MAC mappings

For a local connectivity test, run `ipconfig`, find your actual default gateway, and ping that address. Use your friend's actual IPv4 address when that is the requested test. Do not copy the sample IP addresses from the PDF blindly.

Optional continuous ping: `ping 127.0.0.1 -t`; stop with **Ctrl+C**. For interactive `nslookup`, type `nslookup`, then the domain, then `exit` to leave. A one-line `nslookup mgit.ac.in` is easier to remember.

Ping timeouts can mean ICMP is filtered; they do not alone prove that a website or host is down. A `*` in tracert means the probe did not receive the expected reply before the timeout. Addresses and timings vary with the computer, network and time. These commands were checked against documentation, not executed on the college network; no command output is presented as a live result.

Official command references: ipconfig, ping, tracert, nslookup, arp.

4. Quick viva revision

Question	Short answer
What decides the class of an IPv4 address?	The leading bits, represented by the first-octet range.
What does a subnet mask separate?	The network portion and host portion of an IP address.
How does Dijkstra choose the next node?	Select the unvisited node with the smallest tentative distance.
What is relaxation?	Replace a known distance when a cheaper route is found.
Distance vector vs link state?	Distance vector uses neighbours' distance information; link state uses a network map and shortest-path calculation.
What is a next hop?	The first neighbouring router used to reach a destination.
Why repeat the distance-vector loop?	Updated routes can enable further improvements until the tables converge.
What happens when the leaky bucket is full?	An arrival that cannot fit is dropped.
Is the leaky bucket rate always fully used?	No; it sends less when less data is available.
Why use modulo 26 in Caesar cipher?	To wrap shifts within the 26 uppercase letters.
What is ARP used for?	Resolving a local IPv4 next-hop address to a link-layer/MAC address.

Suggested revision order: Q6 -> Q13 -> Q12 -> Q8 -> Q11 -> Q10 -> Q14. Q8 and Q11 share their main loop, so study that loop once carefully.

Source documents

- Lab syllabus
- Sir's solutions for Q10-Q14